

Object-Oriented Programming in Java

Lecture 3 - Classes and Objects

Emily Lucia Antosch

HAW Hamburg

19.10.2025

Contents

1. Introduction	2
2. Classes and Objects	6
3. Variables and Memory	21
4. Methods	37
5. Constructors	67
6. Class Variables & Class Methods	87
7. License Notice	99

1. Introduction

1.1 Where Are We Now?

- Last time we dealt with the imperative concepts of the Java programming language.
- You can now
 - ▶ use simple data types in Java,
 - ▶ control program flow with control structures and loops, and
 - ▶ convert data types.
- Today we'll cover **Classes and Objects**.

1.1 Where Are We Now?

1. Introduction

1. Imperative Concepts
2. **Classes and Objects**
3. Class Library
4. Inheritance
5. Interfaces
6. Graphical User Interfaces
7. Exception Handling
8. Input and Output
9. Multithreading (Parallel Computing)

1.2 The Goal of This Chapter

- You will implement classes and objects in Java to model real things.
- You will create objects of a class and change their state through operations.
- You will apply additional programming guidelines to improve the quality and maintainability of your code.

2. Classes and Objects

- A **class** is a blueprint for objects. It contains
 - ▶ **Attributes** (data fields) and
 - ▶ **Methods** (operations).
- Together, attributes and methods are called **members**.

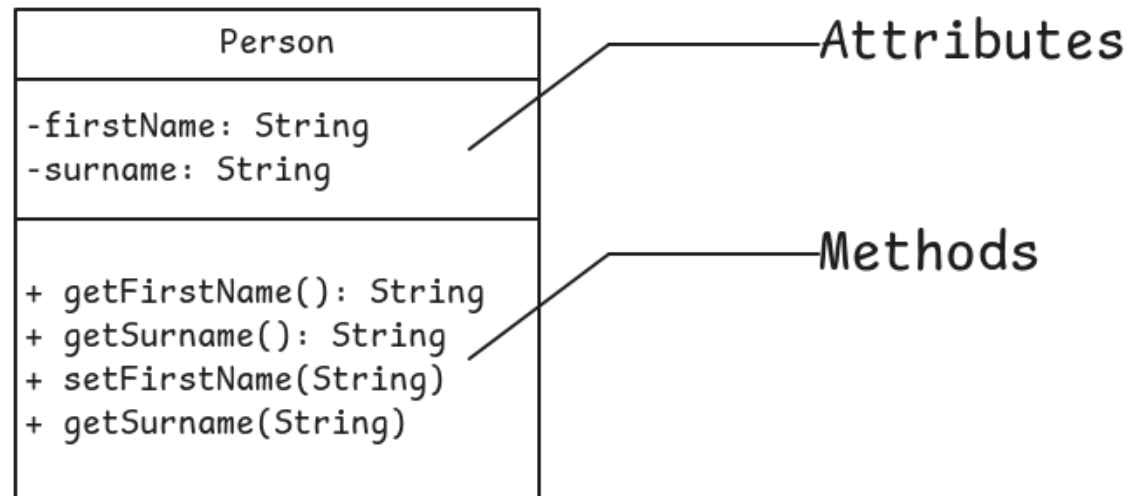


Figure 1: UML notation of a Person class

- Data record of a class created in memory at runtime
- Variables describe the **state** of the object
- Methods describe the **capabilities** of the object
- Terms for variables: **attributes**, **object variables**, **instance variables**

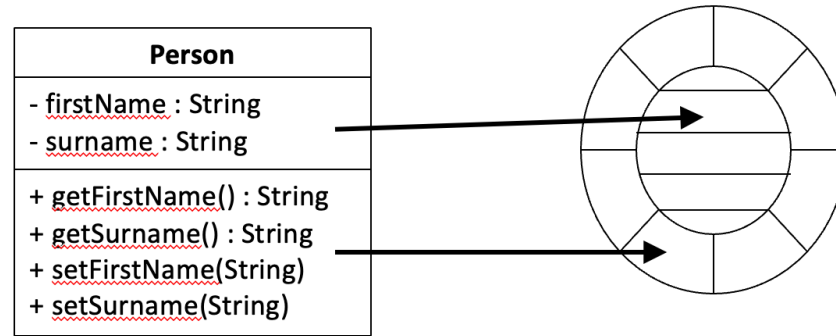


Figure 2: Division of methods and attributes

2.2 Relationship Between Class and Object

Class: Description (“blueprint”) of a data type

- Object of a class: Created element of the data type
- Any number of objects of a class can be created.

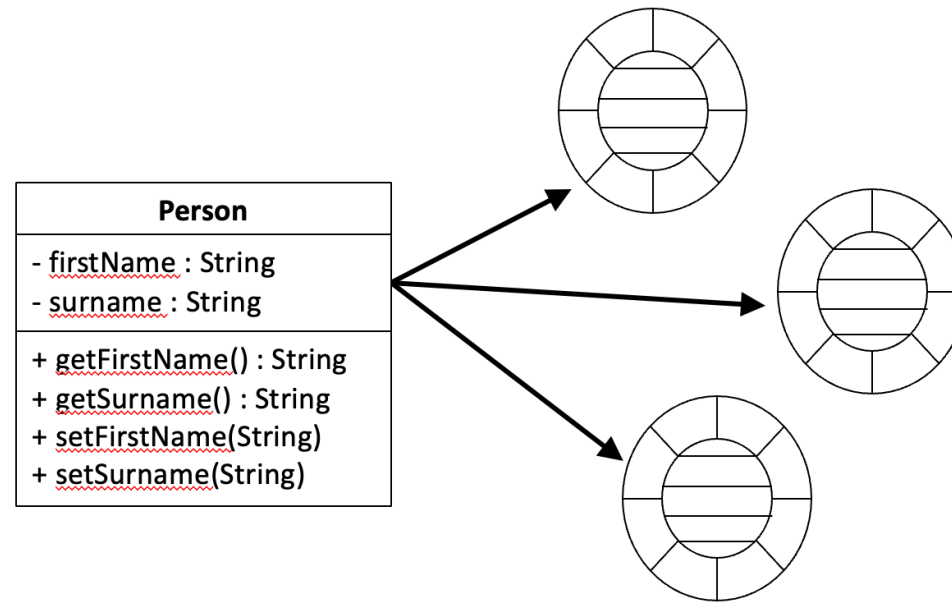


Figure 3: Multiple objects from one class

2.3 Classes in Java

2. Classes and Objects

- Classes can be declared using the following code:

```
1  class ClassName {  
2      Attributes  
3      Methods  
4  }
```



Tip

Create each class in its own file!

Task 1

Let us create this simple class:

- Class Student, described by name, student number and year of enrollment

2.4 Example: Simple Class

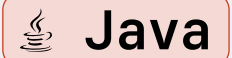
2. Classes and Objects

Task 2

Let us create this simple class:

- Class Student, described by name, student number and year of enrollment

```
1 class Student {  
2     String name;  
3     int matrNumber;  
4     int enrolledYear;  
5 }
```



2.4 Example: Simple Class

- The class has neither methods nor data encapsulation against external influence.

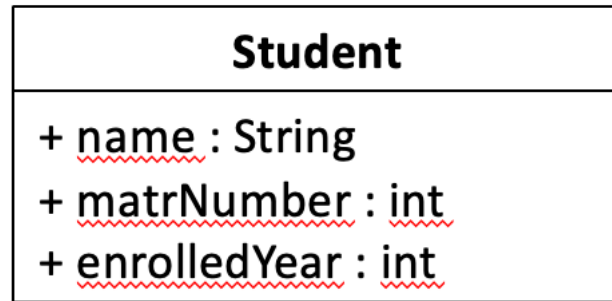


Figure 4: UML representation of the class we just created

2.5 Example: One Class, Many Objects

2. Classes and Objects

- Class (“One class for all students”):
 - ▶ The class is a new data type.
 - ▶ Defines what data describes students
- Objects (“A separate object for each student”):
 - ▶ Objects are instances in memory.
 - ▶ Have the structure of the class, but are filled with data
 - ▶ Any number of objects can be created.

2.5 Example: One Class, Many Objects

2. Classes and Objects

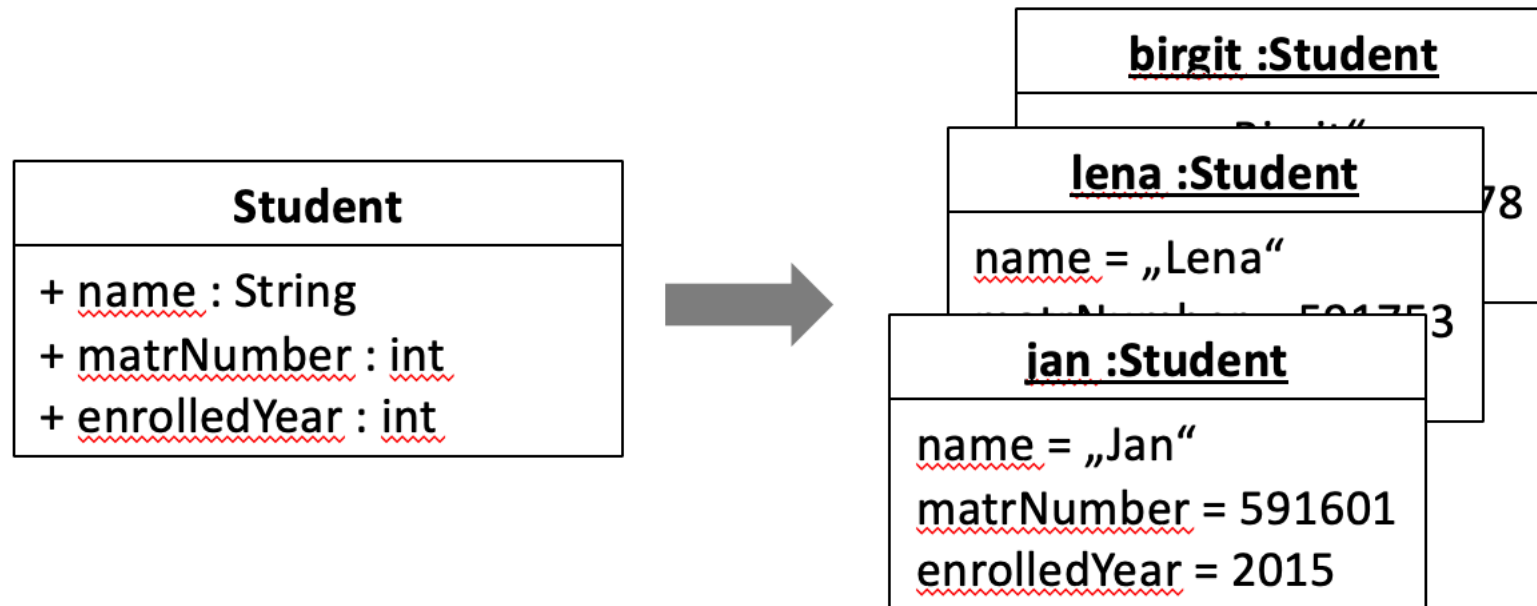


Figure 5: Multiple objects can be created from one class

? Question

What values do the variables `count`, `jan` and `lena` have?

```
1 public class StudentDemo {  
2     public static void main(String[] args) {  
3         int count;  
4         Student lena, jan;  
5     }  
6 }
```

**Java**

? Question

What values do the variables `count`, `jan` and `lena` have?

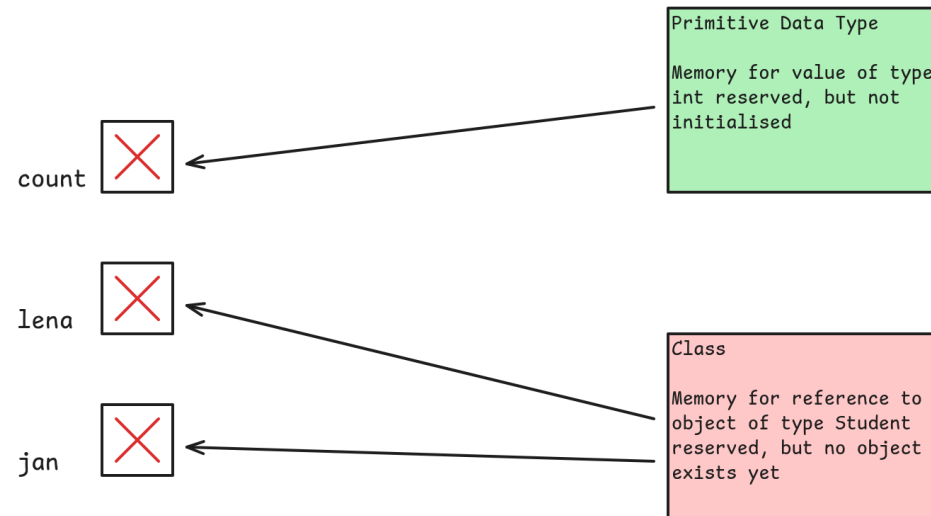
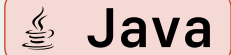


Figure 6: Primitive data types vs. Objects

2.7 Example: new Operator

- Objects are created using the new operator.



```
1 public class StudentDemo {  
2     public static void main(String[] args) {  
3         int count;  
4         Student lena, jan;  
5         lena = new Student();  
6     }  
7 }
```

new-Operator

2.7 Example: new Operator

- Step 1: new operator creates object.
 - ▶ Reserve memory space for object (with object variables).
 - ▶ Initialize object variables with default values (more on this soon).

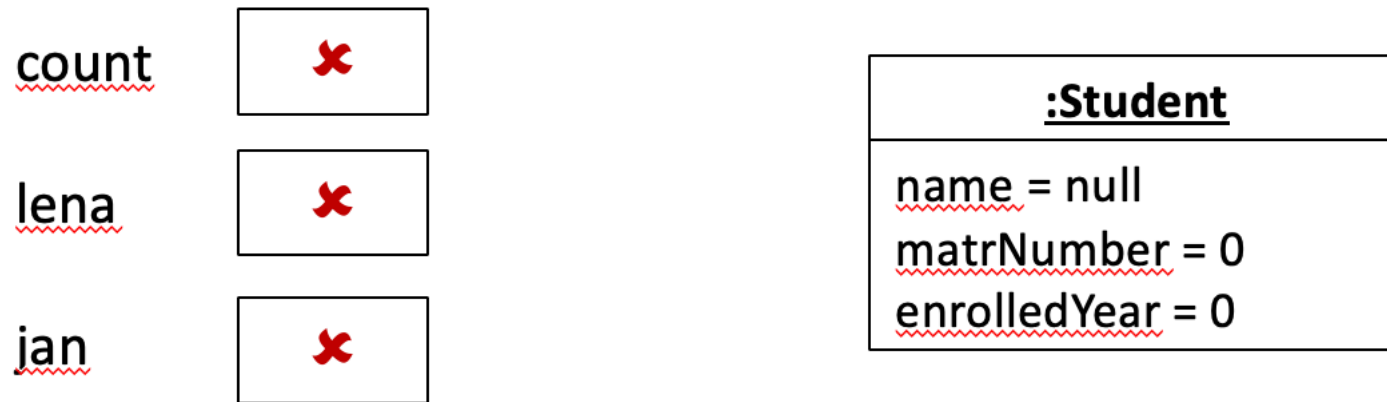


Figure 7: Creating reference with new

2.7 Example: new Operator

- Step 2: Assignment
 - ▶ Writes reference (“address”) of the new object to variable `lena`.
 - ▶ Is independent of the `new` operator and the creation of the object

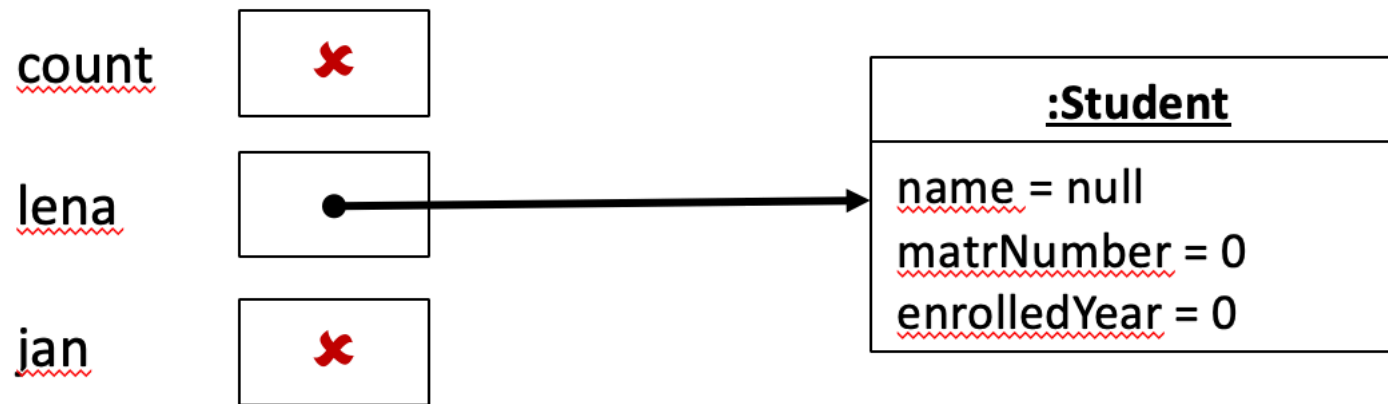


Figure 8: Assignment of reference to variable

3. Variables and Memory



Conclusion

- We have already looked at this:
 - ▶ What are classes and objects?
 - ▶ How do you declare classes?
 - ▶ How do you create objects?
- In the following we want to look at the following aspects:
 - ▶ Access to object variables
 - ▶ Initialization of object variables
 - ▶ Assignment of references

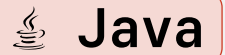
3.1 Objective

- ▶ Automatic memory management

3.2 Access to Object Variables

- Access to object variables is done using the dot operator:

```
1 ObjectReference.Member
```

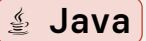


- The `ObjectReference` is a reference to an object that is stored in a variable.
- `Member` is, for example, an attribute/object variable

? Question

What will be output?

```
1  public class StudentDemo1 {
2      public static void main(String[] args) {
3          Student lena = new Student();
4          System.out.println("Enrolled: " + lena.enrolledYear);
5          lena.name = "Lena";
6          lena.matrNumber = 591753;
7          lena.enrolledYear = 2012;
8          System.out.println("Enrolled: " + lena.enrolledYear);
9      }
10 }
```



! Memorize

- **Object/Instance variable:** Declared in class as an attribute of an object.
- **Local variable:** Declared locally (e.g. in method or loop).
- **Reference variable:** Has class as data type, can store reference to object.

- As a reminder:
 - ▶ Local variables are not automatically initialized. (Compiler prevents access.)
 - ▶ Object variables, however, are initialized when an object is created.

Type	Data Type	Initial Value
Integer and Character	byte, short, int, long, char	0
Floating Point	float, double	0.0
Truth	boolean	false
Reference	Any Class	null

Table 1: Value ranges of data types

3.3 Initialization of Classes

- Initial values can also be set in the class itself.

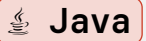
```
1  class Student {  
2      String name = "Unknown";  
3      int matrNumber;  
4      int enrolledYear = 2019;  
5  }
```



? Question

What will be output in the following code?

```
1  public class StudentDemo {
2      public static void main(String[] args) {
3          Student lena = new Student();
4          System.out.println("Name:      " + lena.name);
5          System.out.println("Number:    " + lena.matrNumber);
6          System.out.println("Enrolled:  " + lena.enrolledYear);
7          lena.name = "Lena";
8          System.out.println("Name:      " + lena.name);
9      }
10 }
```



3.4 Assignment of References

3. Variables and Memory

- Assume we have the following state in our code:

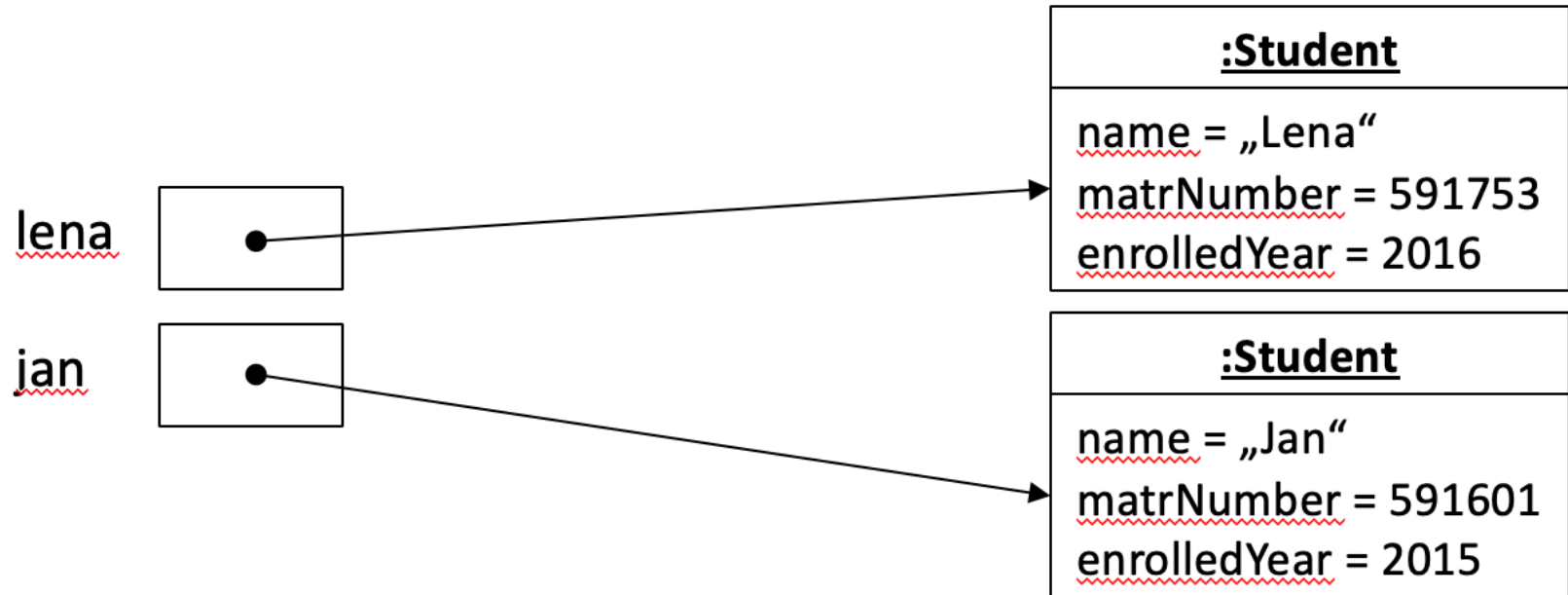
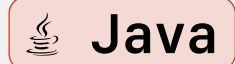


Figure 9: Assignment of references to variables

? Question

What happens now if we add the following code:

```
1 jan = lena;
```



3.4 Assignment of References

3. Variables and Memory

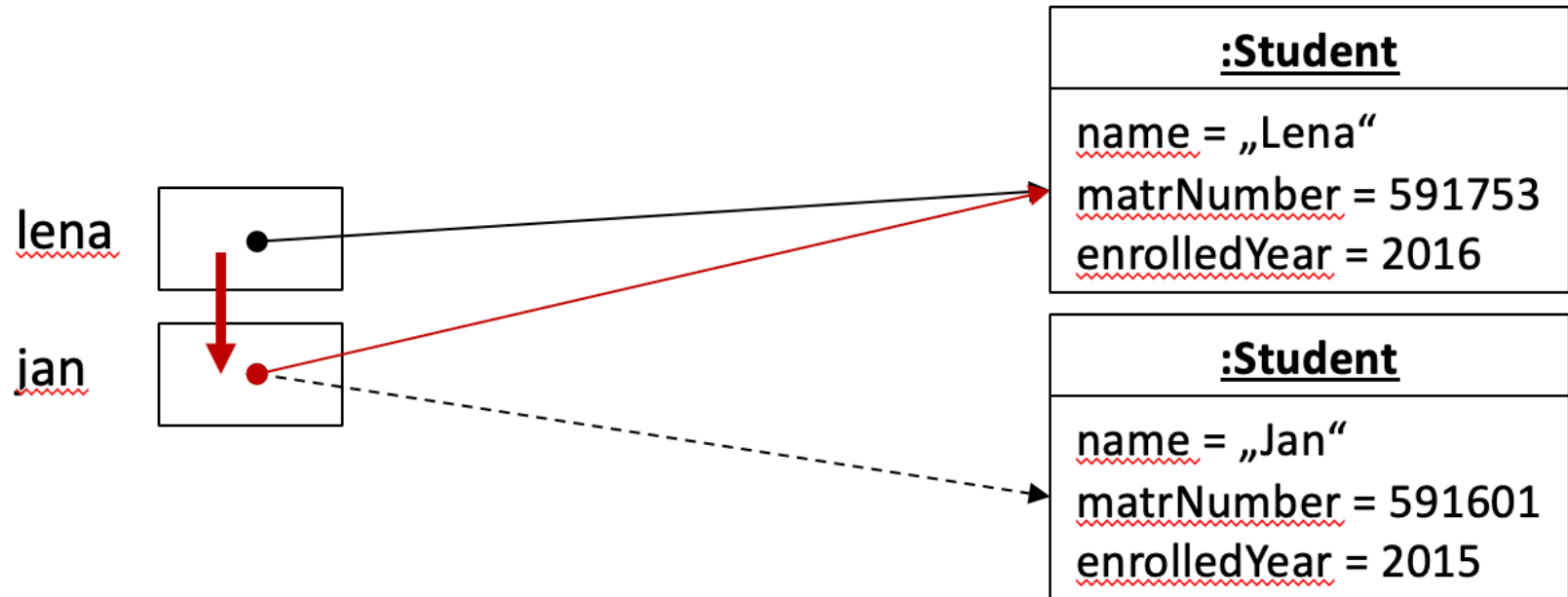


Figure 10: Moving references

? Question

What will be output when you execute the following code afterwards?

```
1 lena.name = "Birgit";  
2 jan.name = "Kai";  
3 System.out.println(lena.name);  
4 System.out.println(jan.name);
```

**Java**

3.4 Assignment of References

- Jan and Lena now reference the **same** object. Changes to values via jan also affect lena.

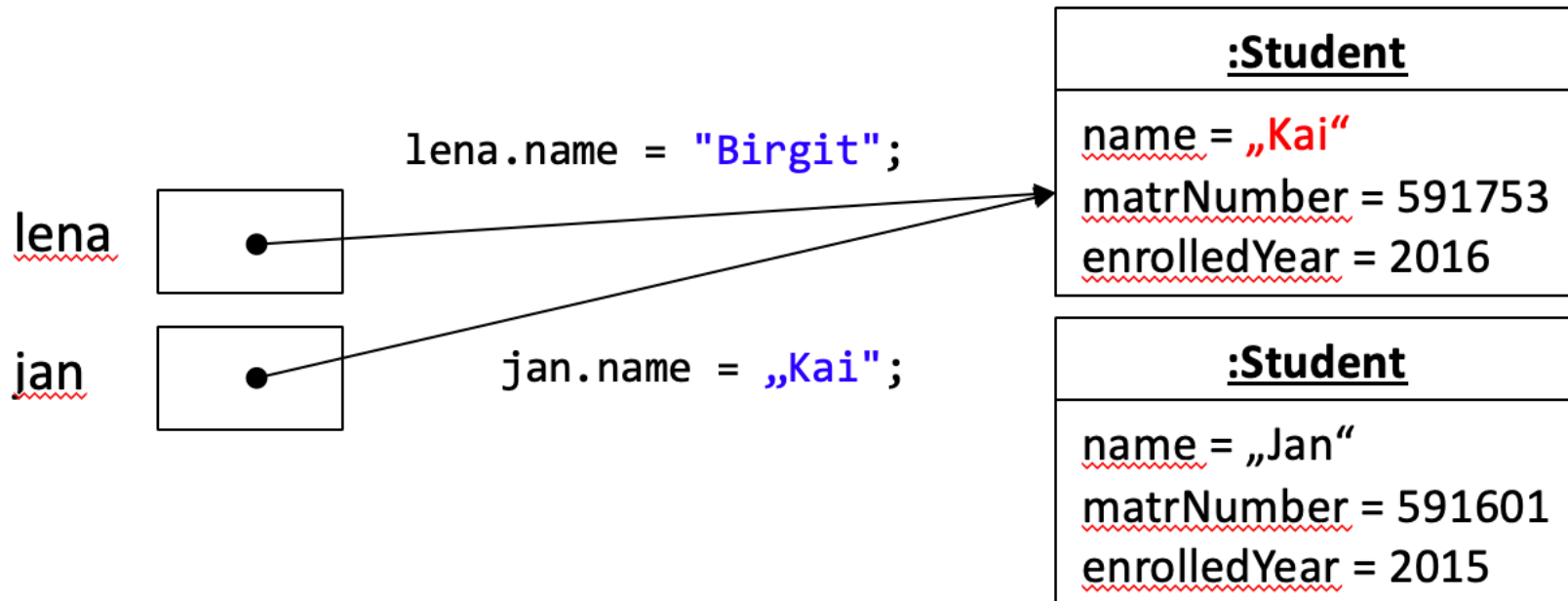


Figure 11: Both variables point to the same reference

3.4 Assignment of References

- The object that was previously referenced via `jan` now has no reference anymore.
- Thus there is **no** way to access the object anymore.
- The **Garbage Collector** will free the memory again using **Reference Counting**.
- There is no `free` or `delete` like in C!

3.4 Assignment of References

3. Variables and Memory

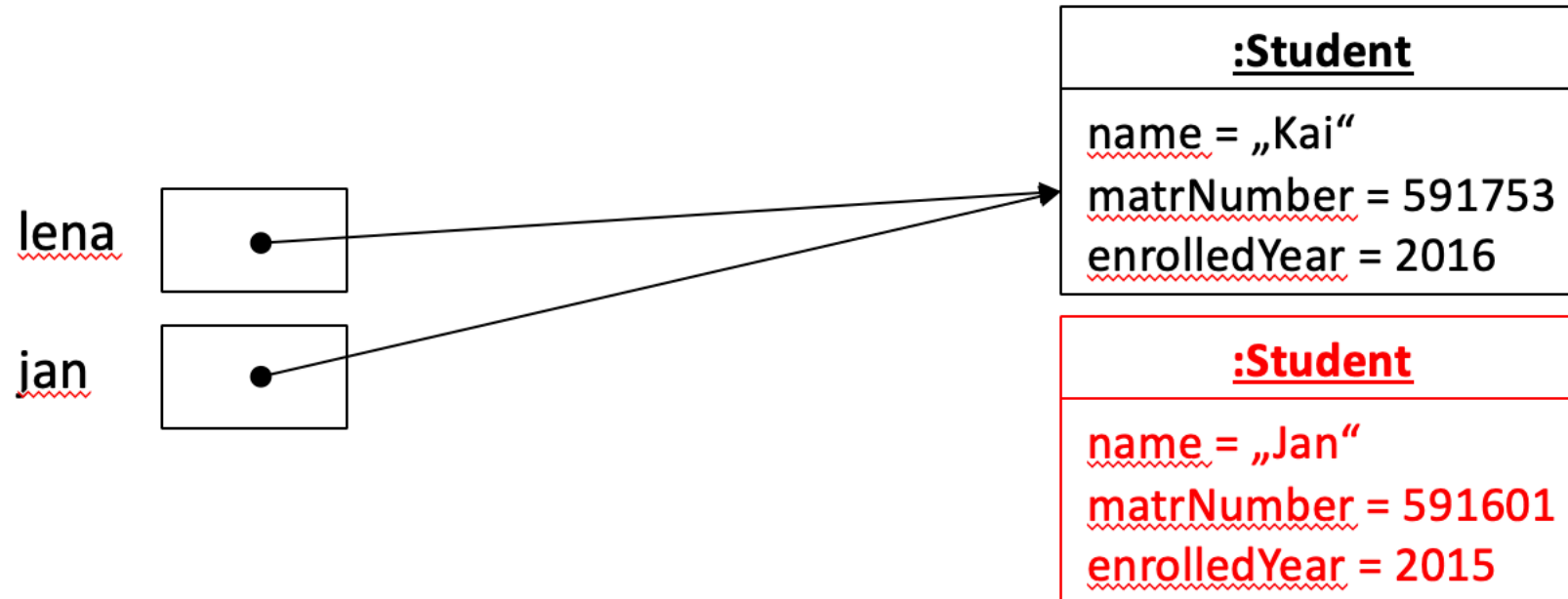


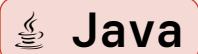
Figure 12: The Garbage Collector frees memory

4. Methods

4.1 Methods: Syntax

- Methods correspond to functions from C, which you already know.

```
1  ReturnType MethodName(Parameters) {  
2    Statement;  
3 }
```



- Return type
 - ▶ Primitive data type, class of an object or `void`.
 - ▶ Return is done as in C using `return`.
- Method name
 - ▶ Any valid identifier (see Chapter 2)
 - ▶ From our coding style: **camelCase** (I also allow **snake_case**)

4.1 Methods: Syntax

- Methods correspond to functions from C, which you already know.

```
1  ReturnType MethodName (Parameters) {  
2    Statement;  
3  }
```



- Parameters
 - ▶ Empty or comma-separated parameters
 - ▶ Each parameter is in the form: datatype identifier
- Call
 - ▶ Method name followed by parentheses
 - ▶ Arguments in the parentheses
 - ▶ Expression is replaced by return value

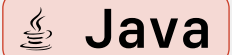
Task 3

Calculate the average of two floating-point numbers.

Task 4

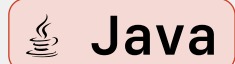
Calculate the average of two floating-point numbers.

```
1 public class MathUtils {  
2     double average(double a, double b) {  
3         return (a + b) / 2.0;  
4     }  
5 }
```



4.2 Methods: Examples

```
1  public class MathUtilsDemo {
2      public static void main(String[] args) {
3          MathUtils math = new MathUtils();
4          double a1 = 3.5, a2 = 7;
5          double mean = math.average(a1, a2);
6          System.out.println(mean);
7          System.out.println(math.average(1.5, 3.2));
8      }
9  }
```



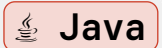
Task 5

Calculate the sum of all digits of an integer.

Task 6

Calculate the sum of all digits of an integer.

```
1 public class MathUtils {
2     int sumOfDigits(int number) {
3         int sum = 0;
4         while(number > 0) {
5             sum += number % 10;
6             number /= 10;
7         }
8         return sum;
9     }
10 }
```



4.2 Methods: Examples

4. Methods

```
1  public class MathUtilsDemo {
2      public static void main(String[] args) {
3          MathUtils math = new MathUtils();
4          System.out.println(math.sumOfDigits(0));
5          System.out.println(math.sumOfDigits(2016));
6      }
7  }
```



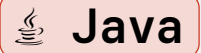
! Memorize

- **Getter:** Method that returns the value of an instance variable
- **Setter:** Method that assigns a (passed) value to an instance variable

4.3 Getters and Setters

4. Methods

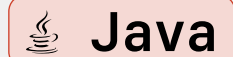
```
1  public class Student {  
2      String name;  
3  
4      void setName(String studentName){  
5          name = studentName;  
6      }  
7  
8      String getName(){  
9          return name;  
10     }  
11  
12 }
```



4.4 Methods: Call-by-Value

- Parameter passing when calling a method:
 - ▶ Basically the value of the variable is passed (“Call by value”).
 - ▶ Not possible to pass a kind of “pointer” to the variable.
 - ▶ In method, the variable used in the method call cannot be changed.

```
1 double square(double a) {  
2     a = a * a; // Local, does NOT modify b in main()  
3     return a;  
4 }
```



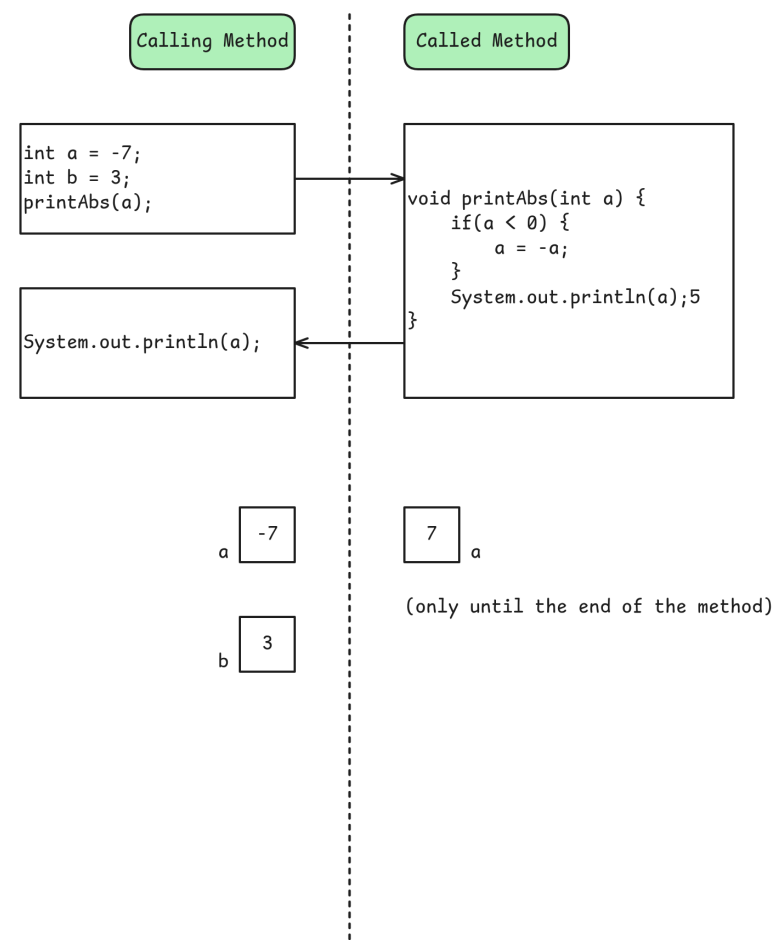
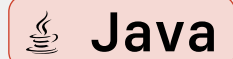


Figure 1: Diagram for Call-by-Value

4.5 Methods: Objects as Parameters

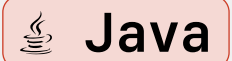
- Reference variables as parameters:
 - ▶ Value (i.e. stored reference) of the passed variable is not changed.
 - ▶ But: The referenced object can be changed!

```
1 public class CallByValueDemo {  
2     static void setNameBirgit(Student student) {  
3         student.name = "Birgit";  
4         System.out.println(student.name);  
5     }
```



4.5 Methods: Objects as Parameters

```
1 public static void main(String[] args) {  
2     Student lena = new Student();  
3     lena.name = "Lena";  
4     setNameBirgit(lena);  
5     System.out.println(lena.name);  
6 }  
7 }
```



4.5 Methods: Objects as Parameters

- Original state in `main()` method:

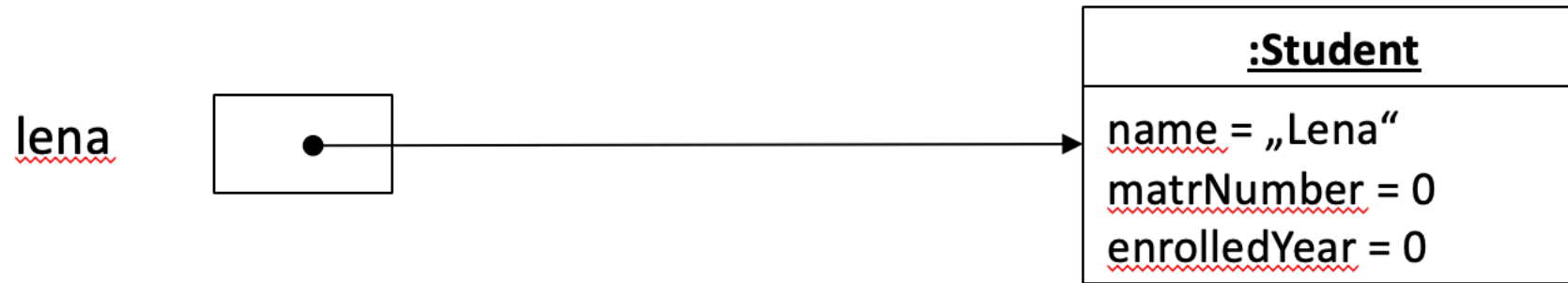


Figure 2: Initial state

4.5 Methods: Objects as Parameters

- Call of the (meaningless) method `setNameBirgit()`:

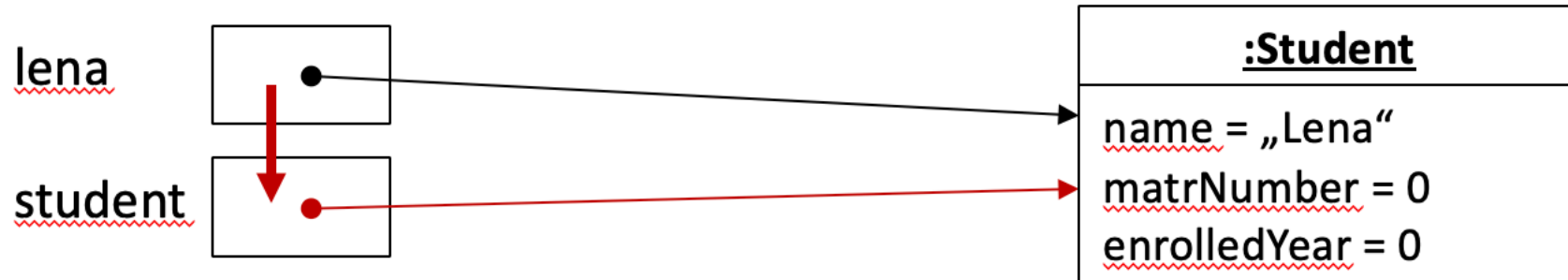


Figure 3: Moving the reference to the method

4.5 Methods: Objects as Parameters

- Modification of the object in the method `setNameBirgit()`:

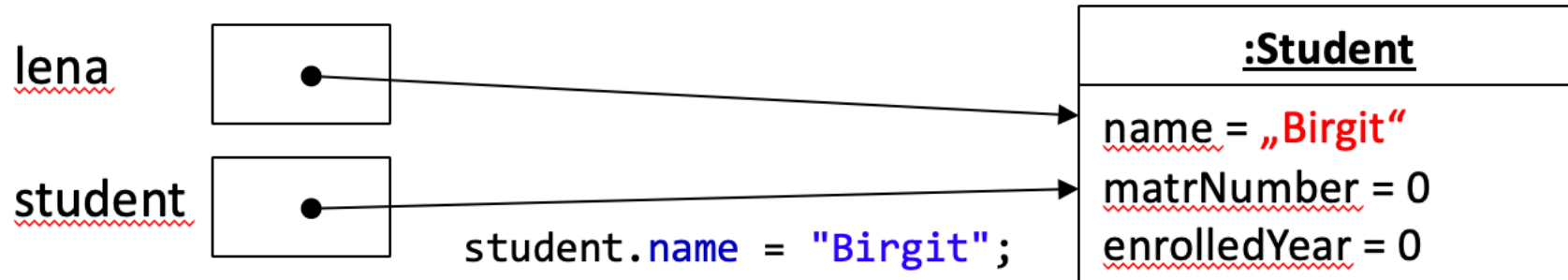
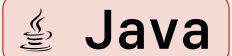


Figure 4: Modification of object in method

? Question

Look at the following code. What is unsightly?

```
1 public class Student {  
2     String name;  
3  
4     void setName(String newName) {  
5         name = newName;  
6     }  
7 }
```



- We would like to name the parameter of the setter name as well!

? Question

Does the following call work?

```
1 void setName(String name) {  
2     name = name;  
3 }
```



- We would like to name the parameter of the setter name as well!

? Question

Does the following call work?

```
1 void setName(String name) {  
2     name = name;  
3 }
```



Java

- No! The compiler would use the local variable in each case.
- How can we access the instance variable?

! Memorize

- Using `this` we can access the current instance we are currently in.

```
1 public class Student {  
2     String name;  
3  
4     void setName(String name) {  
5         this.name = name;  
6     }  
7 }
```



4.6 this Reference

- So
 - ▶ If you want to access a local variable, simply use the identifier of the variable
 - ▶ If you want to access an instance variable, use the `this` reference with the member operator with `..`

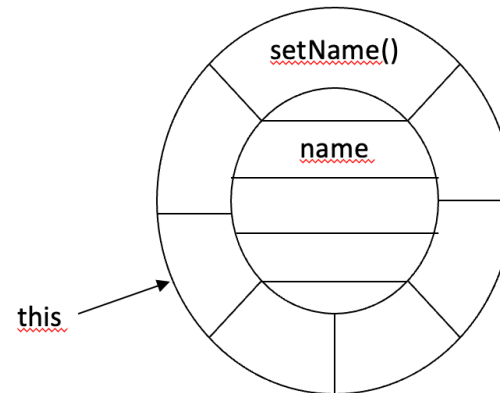
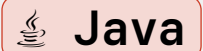


Figure 5: `this` reference

Task 7

Write a method that calculates the maximum of two integers.

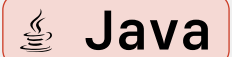
```
1 int max(int a, int b) {  
2     if (a > b) {  
3         return a;  
4     } else {  
5         return b;  
6     }  
7 }
```



Task 8

Write another method in the same class that calculates the maximum of two double numbers.

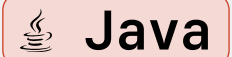
```
1 double max(double a, double b) {  
2     return (a > b) ? a : b; // Compact if/else syntax  
3 }
```



Task 9

Write yet another method in the same class that returns the maximum of three integers.

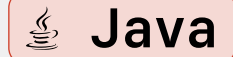
```
1  int max(int a, int b, int c) {  
2      return max(max(a, b), c);  
3  }
```



4.7 Overloading

- Overloaded methods (overloading):
 - ▶ Multiple methods with the same name exist in a class.
 - ▶ Only possible if parameter types are different.
 - ▶ Compiler selects the correct method based on the parameters.

```
1 int max(int, int)
2 double max(double, double)
3 int max(int, int, int)
```

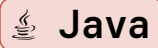


- Note:
 - ▶ Signature: Method name and parameter types
 - ▶ Only data types of parameters relevant (distinction by names not possible)
 - ▶ Distinction by return type is not sufficient (Why?)

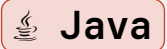
! Memorize

- These are not the same methods!

```
1 int max(int a, int b) {  
2     if (a > b) {  
3         return a;  
4     } else {  
5         return b;  
6     }  
7 }
```



```
1 short max(short a, short b) {  
2     System.out.println("Aaaarrrghhh!");  
3     return 7;  
4 }
```



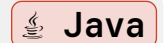
4.8 Methods: Coding Style

- Blank lines should increase code readability.
- A blank line is mandatory between methods or classes (1).
- A blank line is usually placed after declarations (2).
- A blank line is usually placed between logical sections.



Example

```
1  public class MyClass {
2
3      public static int max(int a, int b) {
4          return (a > b) ? a : b;
5      }
6
7      public static void swap(Object a, Object b) {
8          Object temp;
9
10         temp = a;
11         a = b;
12         b = temp;
13     }
14 }
```

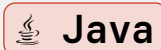


Java

4.8 Methods: Coding Style

- Spaces should increase code readability.
- A space is mandatory between expressions.
- A space is usually placed between operands.

```
1 public static void  
  main(String[] args) {  
2     int a = 5;  
3  
4     for (int i = 1; i < 10; i++) {  
5         a *= i;  
6     }  
7 }
```



```
1 public static void  
  main(String[] args) {  
2     int a=5;  
3  
4     for(int i=1;i<10;i++){  
5         a*=i;  
6     }  
7 }
```



4.9 What a chapter!

- We have accomplished a lot in this section!
 - ▶ Syntax of methods
 - ▶ Parameters (call by value)
 - ▶ this reference
 - ▶ Overloading methods
 - ▶ Coding Style

Let us now look at how to specifically create and initialize objects.

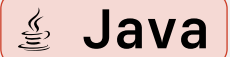
5. Constructors

5.1 Constructors

! Memorize

- Special methods for initializing an object
 - ▶ Are executed when an object is created
 - ▶ Note: Constructors have no return type

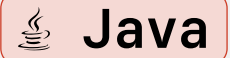
```
1  ClassName(Parameters){  
2  Statements;  
3  }
```



5.1 Constructors

- Using the following code, one could create a constructor for the class Student:

```
1 public class Student {  
2     String name;  
3  
4     Student(String name) { // Attention: No return  
    type  
5         this.name = name;  
6     }  
7 }
```



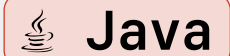
5.1 Constructors

- Default constructor: Constructor with empty parameter list
- Initializes instance variables depending on type with 0, 0.0, false or null



Example

```
1 public class Student {  
2     String name;  
3  
4     Student() {        // Default constructor  
5     }  
6 }
```



5.1 Constructors

- The previous example could also be represented like this:

```
1 public class Student {  
2     String name;  
3  
4     Student() {        // Default constructor  
5         name = null;  
6     }  
7 }
```



5.1 Constructors

- Compiler automatically generates the default constructor under certain conditions
- Simple rules:
 1. If you don't write a constructor for a class:
 - The compiler automatically generates the default constructor
 2. If you write at least one constructor for a class:
 - The compiler does not generate a constructor
 - Only the constructors you implemented exist

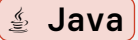
5.2 `this`: Reference vs. Method

- `this` reference (as a reminder):
 - ▶ Use within any (instance) method
 - ▶ Used like a variable
 - ▶ Contains reference to object for which the method was called
- `this()` method
 - ▶ Use only within a constructor.
 - ▶ Is a method call.
 - ▶ `this(parameter list)` calls constructor with matching parameter list.
 - ▶ May only stand as the first statement in the constructor.

5.2 this: Reference vs. Method

5. Constructors

```
1  public class Aircraft {
2      String model, airline;
3      int numberEngines;
4
5      Aircraft() {
6          numberEngines = 1;
7      }
8
9      Aircraft(String model) {
10         this();
11         this.model = model;
12     }
13
14     Aircraft(String model, String airline) {
15         this(model);
16         this.airline = airline;
17     }
18 }
```



? Question

Will the following code compile? What do you think?

```
1 public class Aircraft {  
2     String model;  
3  
4     public static void main(String[] args) {  
5         Aircraft aircraft = new Aircraft();  
6     }  
7 }
```



5.3 Constructors: Examples



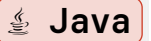
Success

- Yes, it will!
 - ▶ The compiler generates a default constructor because the class does not contain one.
 - ▶ This is then called in the `main()` method!

? Question

Will the following code compile? What do you think?

```
1  public class Aircraft {
2      String model;
3
4      Aircraft(String model) {
5          this.model = model;
6      }
7
8      public static void main(String[] args) {
9          Aircraft aircraft = new Aircraft();
10     }
11 }
```



✗ Error

- No, it will not!
 - ▶ Since the class contains a constructor, the compiler does not generate a default constructor.
 - ▶ The constructor used in the `main()` method therefore does not exist.

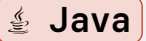
Task 10

- Let us write a class Circle:
 - ▶ Represents a geometric circle
 - Represented by
 - x and y coordinates of the center point
 - Radius r
 - ▶ Constructors:
 - Default constructor creates unit circle around coordinate origin (0 ; 0)
 - Constructor with x, y and radius as parameters
 - Constructor with radius as parameter
 - Constructor with object of class Circle as parameter (creates copy)

5.4 Task: Class Circle

5. Constructors

```
1  public class Circle {
2      double x, y, radius;
3
4      public Circle(double x, double y, double radius) {
5          this.x = x;
6          this.y = y;
7          this.radius = radius;
8      }
9
10     Circle() {
11         this(0.0, 0.0, 1.0);
12     }
13
14     Circle(double radius) {
15         this(0.0, 0.0, radius);
16     }
17
```



5.4 Task: Class Circle

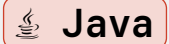
5. Constructors

```
18  Circle(Circle circle) {  
19      this(circle.x, circle.y, circle.radius);  
20  }  
21 }
```


5.4 Task: Class Circle

- The class we just wrote can be created as follows:

```
1 public class CircleDemo {  
2     public static void main(String[] args) {  
3         Circle circle1 = new Circle();  
4         Circle circle2 = new Circle(2.5);  
5         Circle circle3 = new Circle(circle2);  
6         Circle circle4 = new Circle(-1.2, 7.1, 3.0);  
7     }  
8 }
```



Tip

The following menu item in IntelliJ IDEA can save you a lot of work: Code / Generate / Constructor

Task 11

Extend the class with getter and setter methods.



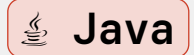
Tip

Via the menu items Code / Generate / Getter and Setter you can save even more time!

5.4 Task: Class Circle

5. Constructors

```
1  double getX() {  
2      return x;  
3  }  
4  
5  // getY() and getRadius() accordingly  
6  
7  void setX(double x) {  
8      this.x = x;  
9  }  
10  
11 void setY(double y) {  
12     this.y = y;  
13 }  
14
```



5.4 Task: Class Circle

5. Constructors

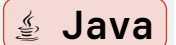
```
15 void setRadius(double radius) {  
16     if (radius >= 0.0) {          // Prevent disallowed data  
17         this.radius = radius;  
18     }  
19 }
```

6. Class Variables & Class Methods

? Question

Do you think you reserve the variable PI only once in memory?

```
1 public class Circle {  
2     double x, y, radius;  
3     final double PI = 3.141592653589793;  
4  
5     double getArea() {  
6         return PI * radius * radius;  
7     }  
8 }
```



✗ Error

- Answer: No, again for each object!

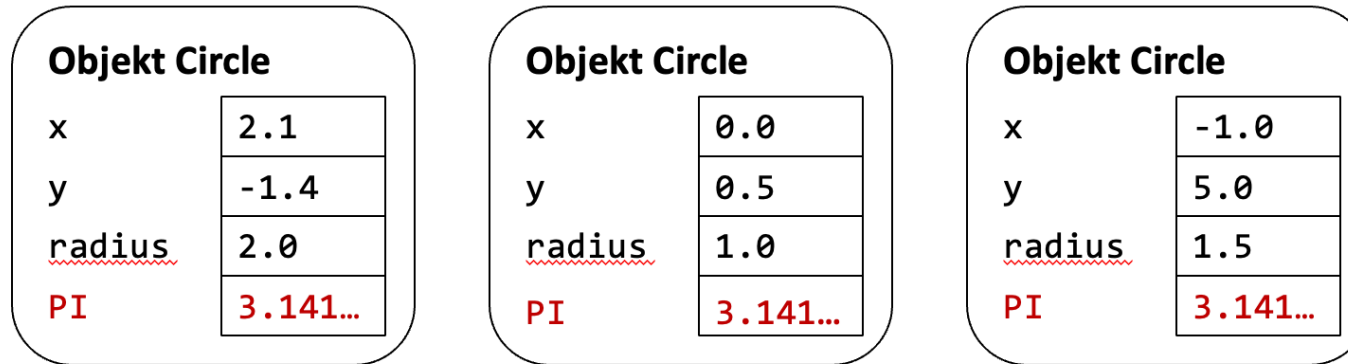
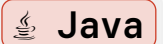


Figure 6: Memory is reserved again for each object.

? Question

What do you think, does the variable `count` count the number of objects?

```
1 public class Circle {
2     double x, y, radius;
3     int count;    // Count number of objects created
4
5     Circle() {
6         radius = 1.0;
7         count++;
8     }
9 }
```



✗ Error

- Answer: No, each object gets a new variable count!

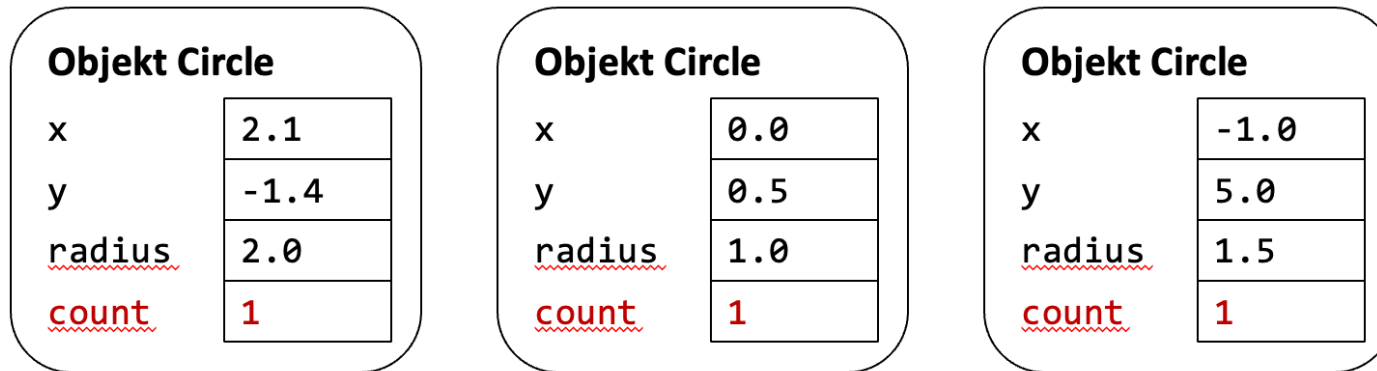


Figure 7: The variable count is created again and again.

- One solution is **class variables**!
- Class variables are created only once for the entire class
- Are not created for a specific (not for each object) object
- Are already created at program start (loading the class)
 - ▶ They exist even when there is (still) no object of the class.
- Syntax: Variable with keyword static

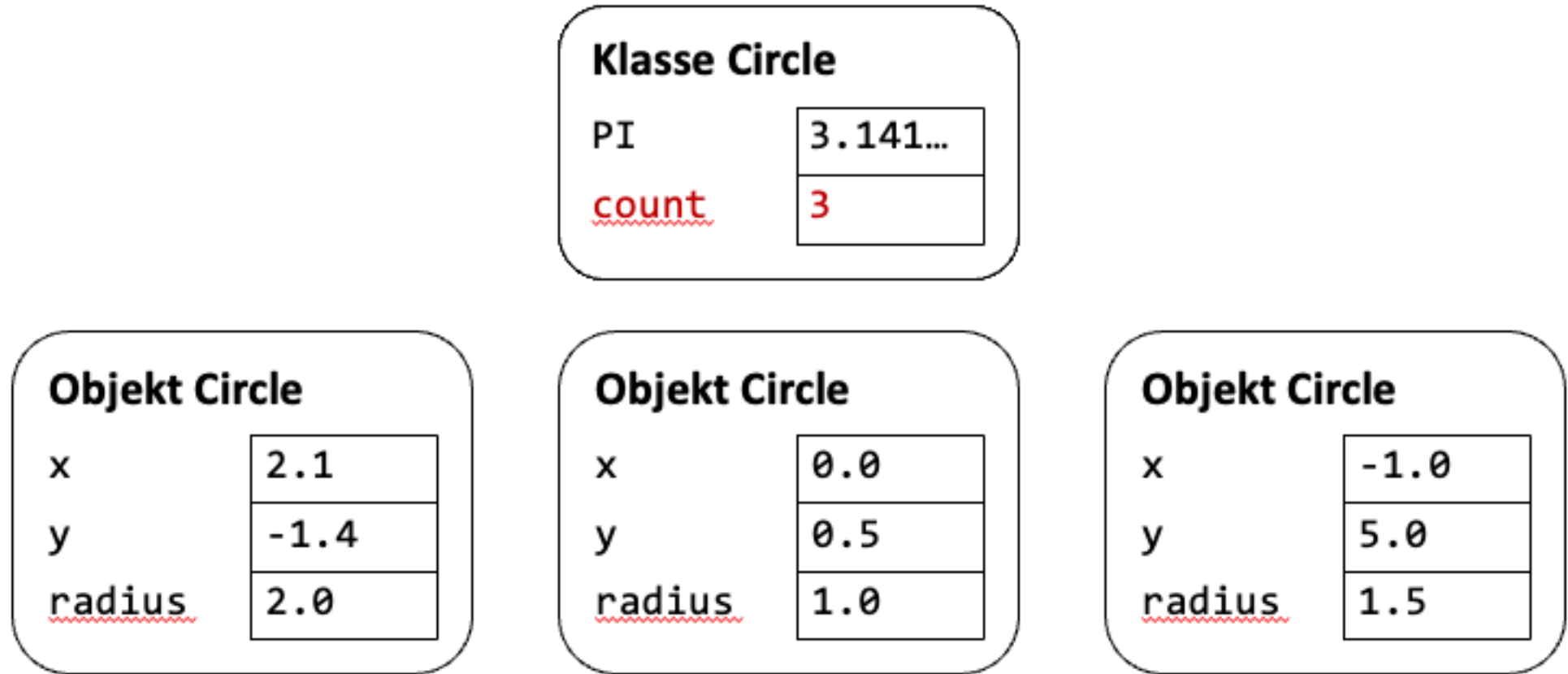


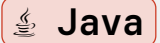
Figure 8: Class attributes in the different objects

6.1 Class Variables

6. Class Variables & Class Methods

- Class Circle with class variables:

```
1  public class Circle {  
2      double x, y, radius;  
3      static final double PI = 3.141592653589793;  
4      static int count;  
5  
6      double getArea() {  
7          return PI * radius * radius;  
8      }  
9  
10     Circle() {  
11         radius = 1.0;  
12         count++;  
13     }  
14 }
```

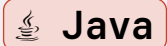


6.1 Class Variables

6. Class Variables & Class Methods

- Access to class variables
 - ▶ Within the method of the class it corresponds to access to instance variables
 - ▶ Outside the class `ClassName.VariableName`

```
1  public class CircleDemo {
2      public static void main(String[] args) {
3          Circle circle1 = new Circle();
4          Circle circle2 = new Circle();
5          Circle circle3 = new Circle();
6
7          System.out.println("Number of objects: " + Circle.count);
8      }
9  }
```

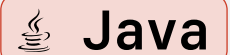


6.2 Class Methods

6. Class Variables & Class Methods

- Completely analogous to class variables:
 - ▶ Class methods are called for a class
 - ▶ Are not called for a specific object
 - ▶ Method becomes class method through keyword static
 - ▶ Can be called without object of the class being created
- Call outside the class:

```
1 ClassName.MethodName
```



! Memorize

- Important consequences:
 - ▶ `this` reference does not exist in class methods
 - ▶ Instance variables do not exist in class methods



Example

- In this example there are two class methods!

```
1  public class Circle {
2      double x, y, radius = 1.0;
3
4      static double getPi() {
5          return 3.141592653589793;
6      }
7  }
8
9  public class CircleDemo {
10     public static void main(String[] args) {
11         System.out.println("Pi: " + Circle.getPi());
12     }
13 }
```



7. License Notice

7.1 Attribution

- This work is shared under the CC BY-NC-SA 4.0 License and the respective Public License
- <https://creativecommons.org/licenses/by-nc-sa/4.0/>
- This work is based off of the work Prof. Dr. Marc Hensel.
- Some of the images and texts, as well as the layout were changed.
- The base material was supplied in private, therefore the link to the source cannot be shared with the audience.